

Supporting CPU Top-down Metrics in PAPI

1st Willow Cunningham
University of Maine
Orono, Maine, USA

2nd Vincent M. Weaver
University of Maine
Orono, Maine, USA
vincent.weaver@maine.edu

Abstract—The Top-down Microarchitecture Analysis method (TMA) provides a simple set of rules that allow any computer program to be methodically analyzed for performance bottlenecks.

The Performance Application Programming Interface (PAPI) library has lacked robust support for TMA; this work describes various enhancements to PAPI to allow better support for this methodology.

The biggest change is adding a PAPI component that supports the special TMA-specific `PERF_METRICS` Model Specific Register (MSR) found on recent Intel processors. Additionally the nuances of accessing the `PERF_METRICS` MSR on heterogeneous multicore processors are discussed and addressed using the Restartable Sequence (rseq) feature of the Linux kernel.

Index Terms—top-down microarchitectural analysis, hardware performance counters.

I. INTRODUCTION

Modern processors contain hardware performance counters: special registers that can measure the low-level architectural behavior of software. These measurements can then be used to optimize the code for faster or more efficient processing. The two most common ways of gathering performance counter data are either total aggregate counts (which give event totals across an entire program run), or else sampled events (where your program is periodically interrupted and counters logged for later statistical analysis). Aggregate counts can be used to find relative performance between runs of your programs. Sampling allows pinpointing where slowdowns are happening. Neither method tells you *why* slowdowns are happening. Traditionally to find slowdown sources on complex modern CPUs you need an expert in low-level computer architecture to analyze your code to find the underlying cause.

In order to make it easier to accurately determine the source of performance bottlenecks Yasin [1] introduced the idea of Top-Down Microarchitecture Analysis (TMA) in 2014. This provides a simple, methodical approach to iteratively narrowing down the specific hardware performance bottlenecks of a program. By using a curated set of specific events it allows finding the root cause of slowdowns in modern processor pipelines rather than having to make educated guesses based on the sampled or aggregate results.

While initially proposed for Intel processors, the same techniques can be applied to other architectures. This includes AMD [2], RISC-V [3], ARM [4], [5], and various other computing setups commonly found in HPC environments [6]. In addition to CPUs TMA can also be applied to GPUs [7] however in this document we focus on CPU support.

The Performance API (PAPI) [8] is a cross-platform library for measuring performance counter results, commonly used in the high performance computing (HPC) field. In this work we describe various enhancements made to PAPI to make gathering TMA measurements easier. Our goal is to provide a common interface for easily obtaining TMA data across a wide variety of systems.

II. RELATED WORK

There are a wide variety of available performance measurements tools. Many of them have various level of support for gathering Top Down results.

Any tool that is able to collect HPCs can be used to acquire TMA metrics. As discussed in Section IV this has always been true for PAPI as well. However, the process that one must follow to extract TMA metrics from raw HPCs can be confusing, obscure, and microarchitecture specific. To this end some tools provide explicit, and to varying degrees cross-platform support for collecting TMA metrics.

The Linux `perf` tool provides explicit support for top-down metrics on modern Intel processors and a few Arm processors via the `--topdown` flag:

```
perf stat --topdown /bin/ls
```

Recent AMD processors are also supported, but they are called Pipeline events instead:

```
perf stat -M PipelineL1 /bin/ls
```

Once the proper events to use are known for the processor of interest `perf` or any other tool can measure them. There are often subtleties that might require extra support.

For Intel processors in particular, the `pmu-tools` project [9] provides scripts that streamline the process of collecting TMA metrics and drilling down through the hierarchy to locate bottlenecks. Separately, Intel's VTune Profiler [10] provides a graphical user interface for using TMA to analyze performance bottlenecks in programs run on Intel processors. Arm provides the `topdown-tool` in their Telemetry Solution repository [11], which is similar in function to `pmu-tools`.

While each of these tools provide explicit support for TMA to varying degrees, unlike PAPI none are capable of “caliper” programs to collect metrics for arbitrary blocks of code within a program.

III. TMA BACKGROUND

The Top-down Microarchitecture Analysis method defines a methodical, hierarchical approach to using certain Perfor-

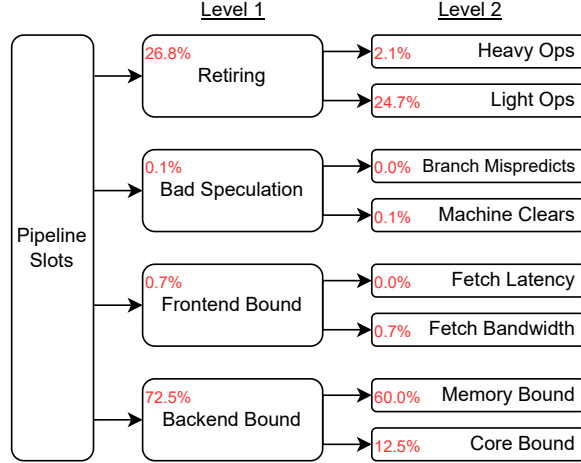


Fig. 1. The Hierarchy of Level 1 and 2 Top-down Metrics, Including Typical Metric Percentages for `mbw`, a Memory Intensive Workload.

mance Monitoring Unit (PMU) events in order to discover critical hardware performance bottlenecks for any application. Core to the TMA are the concepts of pipeline slots and hierarchical categorization. Pipeline slots are defined to be the product of the micro-architecture pipeline width and the number of cycles that occur for a given measurement period. The behavior of each slot is measured using PMU events and categorized into a hierarchical tree of successively narrower categories, or metrics. These metrics are presented as percentages of the total number of slots recorded over a given measurement period.

At the first level, slots are split into four different metrics. The “Retiring” metric records slots used to successfully retire μ Ops, the “Bad Speculation” metric records slots that were wasted due to speculatively executed instructions which could not be retired, the “Frontend Bound” metric records slots wasted due to the front end failing to issue enough μ Ops in a cycle, and the “Backend Bound” metric records slots wasted due to the pipelines having to wait on the back end of the processor.

A. TMA Example

Figure 1 shows a breakdown of the first two levels of the TMA hierarchy with example recorded metric percentages from a run of `mbw` [12] which is a simple memory-intensive workload.

The `mbw` program copies a large array using three different strategies (`memcpy`, `mcblock`, and `dumb`) in order to benchmark a system’s memory bandwidth. It serves here as an example of a memory-intensive application. The results are gathered on the Intel Raptor Lake M1 machine described by Table I with problem size $M = 1000$, averaged over $N = 500$ runs.

TABLE I
TESTING SYSTEMS

Code name	Raptor Lake (M1)
Processor	13th Gen Intel(R) Core(TM) i7-13700
P-cores	8 Golden Cove cores (2 threads/core)
E-cores	8 Gracemont cores (1 thread/core)
Operating system	Debian GNU/Linux 6.11.10

Code name	Raptor Lake (M2)
Processor	14th Gen Intel(R) Core(TM) i7-14700
P-cores	8 Golden Cove cores (2 threads/core)
E-cores	12 Gracemont cores (1 thread/core)
Operating system	Ubuntu GNU/Linux 6.8.0

Code name	Sapphire Rapids (M3)
Processor	Intel(R) Xeon(R) Gold 6430
Cores	32
Operating system	Oracle Linux 4.18.0

B. Higher Level TMA

Since the first release of the TMA, subsequent versions have defined more metrics, filling out further levels in the tree. As of TMA 5.01, there are a total of 6 levels of TMA metrics with each level becoming more specific to hardware details and less generalizable. Because this paper focuses only on accessing the Level 1 and Level 2 metrics available in the `PERF_METRICS` MSR, further discussion on Levels 3-6 is left for future work.

C. TMA via Standard Events

Usually the events needed for TMA are just regular performance events available through the standard performance counter interfaces. There are a few challenges that a tool like PAPI needs to take into account before supporting them.

1) *Event Selection*: Ideally the vendor supplies a list of which events to use; that is because modern systems can have excessively large amounts of events and it can be hard to know which ones are appropriate. For example: currently Intel’s Sapphire Rapids architecture has 396 documented events for the core PMU. This can get even worse on heterogeneous multicore processors as individual core types may have PMUs with different events. The Raptor Lake architecture’s performance cores (p-cores) have 311 events and its efficiency cores (e-cores) have 182 events, bringing the total number of documented on-core events to 493. When faced with such a massive set of HPCs to choose from it can be difficult for a performance analyst to know where to start, especially as each new processor requires learning the behaviors of a whole new set of events.

Another issue is determining the events to use. In this example we are looking at the “Pipeline Utilization Analysis” (PUA) events on AMD Zen5 [13]. PUA events are the AMD equivalent of topdown events. As can be seen in Table II the AMD manual lists the events to use by number. The corresponding events in the AMD manual have names applied which in theory is better than using raw hex values when using tools. However as can be seen the name supplied by the Linux/perf tool json files does not always match the names

TABLE II
AMD ZEN5 LEVEL 1 TOPDOWN/PIPELINE UTILIZATION ANALYSIS EVENTS USED

Top Down Event	Raw Event	perf/json Name	libpfm4/PAPI Name
Total Dispatch Slots	0x430076	ls_non_halted_cyc	CYCLES_NOT_IN_HALT
Frontend Bound	0x1004301a0	de_no_dispatch_per_slot. no_ops_from_frontend	DISPATCH_STALLS_1:FE_NO_OPS
Bad Speculation	0x4307aa 0x4300c1	de_src_op_disp.all ex_ret_ops	OPS_SOURCE_DISPATCHED_FROM_DECODER:DECODER:OPCACHE RETIRED_OPS
Backend Bound	0x100431ea0	de_no_dispatch_per_slot. backend_stalls	DISPATCH_STALLS_1:BE_STALLS
SMT contention	0x1004360a0	de_no_dispatch_per_slot. smt_contention	DISPATCH_STALLS_1:SMT_CONTENTION
Retiring	0x4300c1	ex_ret_ops	RETIRED_OPS

from the manual. And to be even more confusing the libpfm4 name used by PAPI does not match the other two. This can make it difficult when picking which events to assign.

This specific Zen5 example has an additional issue: the event specified for the “bad speculation” metric in the PUA documentation uses a bit marked “reserved” later in the same manual. The `perf json` file actually defines this event, but libpfm4 does not, making it tricky to add this event to an eventset in PAPI.

2) *PAPI Preset Event Selection*: Once the events to be used for a TMA run have been selected it seems like it should be straightforward to just create PAPI pre-defined event sets for the various Level 1 and Level 2 groups.

It turns out though this gets more complex. On some architectures the events to use can vary by CPU model/stepping. This is a finer grain distinction than PAPI normally makes when doing CPU detection and the current PAPI presets do not support this. We are investigating the best way to handle this, possibly a PAPI component would be the best way to support the proper detection and use of TMA events.

3) *Sufficient Event Slots*: Another issue complicating TMA support is being able to measure all of the desired events simultaneously. For example, measuring the events for Level 1 on AMD Zen5 processors requires measuring 6 events. The Zen5 hardware supports this many simultaneous measurements, but unfortunately the Linux Software Watchdog by default steals one of the event slots meaning it is not possible to gather full Level 1 TMA without disabling this.

To avoid this you either have to carefully select events to make sure they fit, disable the software watchdog, or else try to use some sort of multiplexing. An additional way to do this is have specific hardware support for TMA, we will discuss this in the next subsection.

D. TMA Hardware Support

Since the introduction of Intel’s 10th generation Ice Lake microarchitecture, Intel processors support the `PERF_METRICS` model specific register (MSR). This MSR is a special register that allows collecting 8 or more TMA events with a single register read, rather than using the more standard PMU interface that would require co-ordinating a separate read for each event.

Earlier processors support only the four Level 1 top-down metrics, while processors from the 12th generation Alder Lake architecture and onwards provide direct access for four of the eight Level 2 top-down metrics as well, enabling the derivation of all Level 1 and 2 metrics with only one read. The contents of the `PERF_METRICS` MSR are shown in Table III.

Each 8-bit field represents an integer fraction of slots that have been identified as belonging to the given metric. (Note: while the Intel Software Developer Manual (SDM) calls bits 55-48 and 39-32 “Frontend latency” and “Multi uops” respectively, elsewhere in their documentation these are labeled “Fetch latency” and “Heavy ops”. For the sake of consistent language, this paper uses the latter.)

Unfortunately, the availability of the `PERF_METRICS` MSR is a bit more complicated on heterogeneous processor families like Raptor Lake. As of the time of writing, each of Intel’s heterogeneous processors only support the MSR on their p-cores. Extra care must be taken to ensure that there are no attempts to read the `PERF_METRICS` MSR when on an unsupported e-core or it will produce a general protection fault, leading to the measurement program crashing. Two solutions to this are described in Section VI.

To complement the `PERF_METRICS` MSR, architectures supporting it also implement a counter to measure the cumulative number of clocked pipeline slots in fixed counter 3 as the event `TOPDOWN.SLOTS`. Its value can be approximated as the product of the processor pipeline width and the number of unhalted cycles.

Prior to the introduction of the `PERF_METRICS` MSR and fixed counter 3, Intel derived the slots metric in this way on some machines. For example, on Ivy Bridge they use `CPU_CLK_UNHALTED.THREAD*4` as discussed in [1]. On later microarchitectures like Sky Lake that still predated the `PERF_METRICS` MSR, the general purpose event `UOPS_RETIRED.RETIRE_SLOTS` is used instead [15].

IV. PAPI IMPLEMENTATION OF INTEL TMA HARDWARE

The code base of PAPI is split into a core system that defines the API for caliper programs, and components which utilize the interface to collect specific kinds of information. The two functions exposed to the user which enable performance caliper are `PAPI_start()` and `PAPI_stop()`, which can be used to measure particular events in an `EventSet`

TABLE III
THE PERF_METRICS MSR, COMPILED FROM TABLES 2-44 AND 2-54 OF THE INTEL SDM VOLUME 4 [14].

Architecture	bits 63-56	bits 55-48	bits 47-40	bits 39-32	bits 31-24	bits 23-16	bits 15-8	bits 7-0
10th gen +	Reserved				Backend bound	Frontend bound	Bad spec.	Retiring
12th gen +	Memory bound	Fetch latency	Branch miss	Heavy ops	Backend bound	Frontend bound	Bad spec.	Retiring

as shown in Listing 1. `PAPI_start()` takes an `EventSet`, which is a structure containing a set of events to measure during the execution of code between the `PAPI_start()` and `PAPI_end()` calls.

Listing 1. Example C Code Demonstrating Code Caliperizing With PAPI
#include <papi.h>

```
int main() {
    // not shown:
    // initialize PAPI...
    // set up the EventSet...

    PAPI_start(EventSet);
    my_work_function();
    PAPI_stop(EventSet, values);

    // measurements for my_work_function()
    // are now stored in the values array!
}
```

The PAPI `perf_event` component uses the Linux `perf_event` subsystem [16] to collect hardware performance counters provided by the processor's micro-architecture. In theory this component can be leveraged to collect TMA metrics through HPCs such as `TOPDOWN.SLOTS` and `TOPDOWN.BAD_SPEC_SLOTS` supported by Intel's Raptor Lake architecture [15]. However, there are a few key issues which make this approach unwieldily to the point of being infeasible.

The Linux `perf_event` subsystem has a concept of event groups and group leaders. In order to read top-down PMU events like `TOPDOWN.BAD_SPEC_SLOTS`, the special `TOPDOWN.SLOTS` event must be the group leader. This fact is obscure, but not impossible to discover. Once the user is aware of this particularity unique to top-down PMU events, they must then figure out how to make `TOPDOWN.SLOTS` the group leader of a PAPI `EventSet`. Currently adding `TOPDOWN.SLOTS` to the `EventSet` first will make it the group leader, but this is an implementation detail and to discover this the user must either make an educated guess or be familiar with the code of the PAPI `perf_event` component.

To make matters worse, on x86 processors the `perf_event` component by default uses the `rdpmc` instruction to read PMU events. As shown by Liu and Weaver [17], this significantly lowers the performance overhead of the component. However in the special case of top-down PMU events like `TOPDOWN.BAD_SPEC_SLOTS` the behavior of the event changes based on whether it is read using `perf_event_open` or the `rdpmc` instruction. If

read with `perf_event_open` the event will report the total number of slots wasted due to bad speculation as an unsigned 64-bit integer. This is the expected behavior. However when read with `rdpmc` it instead returns the contents of the `PERF_METRICS` model-specific register (MSR) shown in Table III. Configuring PAPI to not use low-latency reads with the `--disable-perfevent-rdpmc` option ensures the events behave as expected, but loses the significant performance gains of using `rdpmc`.

Previously, there have existed two incompatible methods to acquire TMA metrics in PAPI: with `rdpmc` disabled, and with `rdpmc` enabled. With `rdpmc` disabled, which we will refer to as Method A, it is possible to add enough events directly measuring HPCs to the `EventSet` to calculate the TMA metrics.

1) *Previous PAPI TMA Method A:* Finding these events for a given processor is unfortunately not intuitive. For example, on the Alder Lake architecture the `TOPDOWN` category of HPCs provides events corresponding to the backend bound, bad speculation, branch mispredicts, and memory bound metrics. To collect all 12 metrics, at least one more level 1 metric and two more level 2 metrics need corresponding HPCs. For Alder Lake, these are `INST_RETIRED.ANY`, `UOPS_RETIRED.HEAVY`, and `IDQ_UOPS_NOT_DELIVERED.CYCLES_0_UOPS_DELIV.CORE`. The latter's value must be multiplied by the pipeline width of the processor, which is 6 in this case.

An example of what this looks like in code is provided in Listing 2. Error handling is excluded from this example in the service of conciseness.

Listing 2. Example C Code for collecting TMA metrics in PAPI With `rdpmc` Disabled.

#include <papi.h>

```
int main() {
    int EventSet;
    long long values[8];

    PAPI_library_init(PAPI_VER_CURRENT);

    PAPI_create_eventset(&EventSet);
    PAPI_add_named_event(EventSet,
        "TOPDOWN: SLOTS");
    PAPI_add_named_event(EventSet,
        "TOPDOWN: RETIRING_SLOTS");
    PAPI_add_named_event(EventSet,
        "TOPDOWN: BAD_SPEC_SLOTS");
    PAPI_add_named_event(EventSet,
        "TOPDOWN: BACKEND_BOUND_SLOTS");
    PAPI_add_named_event(EventSet,
```

```

    "UOPS_RETIRED:HEAVY");
PAPI_add_named_event(EventSet,
    "TOPDOWN:BR_MISPREDICT_SLOTS");
PAPI_add_named_event(EventSet,
    "IDQ_UOPS_NOT_DELIVERED:");
    "CYCLES_0_UOPS_DELIV_CORE");
PAPI_add_named_event(EventSet,
    "TOPDOWN:MEMORY_BOUND_SLOTS");

// caliper my_work_function...
PAPI_start(EventSet);
my_work_function();
PAPI_stop(EventSet, values);

// extract metric percentages...
double retiring = (double)values[1]
    / values[0]*100.0;
double bad_spec = (double)values[2]
    / values[0]*100.0;
double frontend = (double)(values[0]-
    values[1]-values[2]-values[3])
    / values[0]*100.0;
double backend = (double)values[3]
    / values[0]*100.0;
double heavy_ops = (double)values[4]
    / values[0]*100.0;
double light_ops = retiring-heavy_ops;
double br_miss = (double)values[5]
    / values[0]*100.0;
double mchn_clear = bad_spec-br_miss;
double fetch_lat = (double)values[6]
    / values[0]*100.0*6;
double fetch_band = frontend-fetch_lat;
double mem_bound = (double)values[7]
    / values[0]*100.0;
double core_bound = backend-mem_bound;
}

```

There are three main drawbacks to Method A. First, there is the requirement that PAPI be configured in a non-standard manner to disable the use of the `rdpmc` instruction by the `perf_event` component. This means that for systems with PAPI installed and managed by the administrator, the user must go through the hurdle of compiling PAPI locally. The second drawback for this method is that simply understanding how to *utilize* TMA is insufficient — the user must additionally research the *implementation* of TMA to discover which HPCs to add to the EventSet in order to derive accurate TMA metrics.

Finally, many processors cannot actually support measuring 12 HPCs at once. For example, the Raptor Lake architecture has 6 general purpose counters in the PMU. This means that the PAPI `perf_event` component must multiplex the counters which leads to less accurate results. This can be mitigated by only collecting a portion of the TMA metrics at a time, though this of course increases the number of recompilations

and reruns of the caliper program. The overall effect of these issues is a significant burden of knowledge and time on the user.

2) *Previous PAPI TMA Method B*: The second pre-existing method for collecting top-down metrics is to configure PAPI with the default settings, thereby enabling the `perf_event` component to use `rdpmc`. Any HPC in the `TOPDOWN` category will then retrieve the contents of the `PERF_METRICS` MSR instead of an integer representing the number of recorded slots for a given metric. While this behavior is obscure and unintuitive, it does result in more concise user code, as shown in Listing 3.

Listing 3. Example C Code for collecting TMA metrics in PAPI With `rdpmc` Enabled.

```

#include <papi.h>

int main() {
    int EventSet;
    long long values[8];

    PAPI_library_init(PAPI_VER_CURRENT);

    PAPI_create_eventset(&EventSet);
    PAPI_add_named_event(EventSet,
        "TOPDOWN:SLOTS");
    PAPI_add_named_event(EventSet,
        "TOPDOWN:RETIRING_SLOTS");

    // caliper my_work_function...
    PAPI_start(EventSet);
    my_work_function();
    PAPI_stop(EventSet, values);

    // extract metric percentages...
    double retiring = (double)(
        values[1]>>(0*8) & 0xff)
        / 0xff * 100.0;
    double bad_spec = (double)(
        values[1]>>(1*8) & 0xff)
        / 0xff * 100.0;
    double frontend = (double)(
        values[1]>>(2*8) & 0xff)
        / 0xff * 100.0;
    double backend = (double)(
        values[1]>>(3*8) & 0xff)
        / 0xff * 100.0;
    double heavy_ops = (double)(
        values[1]>>(4*8) & 0xff)
        / 0xff * 100.0;
    double light_ops = retiring-heavy_ops;
    double br_miss = (double)(
        values[1]>>(5*8) & 0xff)
        / 0xff * 100.0;
    double mchn_clear = bad_spec-br_miss;
    double fetch_lat = (double)(
        values[1]>>(6*8) & 0xff)

```

```

    / 0xff * 100.0;
double fetch_band = frontend - fetch_lat;
double mem_bound = (double)(
    values[1] >> (7*8) & 0xff)
    / 0xff * 100.0;
double core_bound = backend - mem_bound;
}

```

There is a problem with this approach that makes it unusable for collecting all level 1 and level 2 TMA metrics. The PAPI `perf_event` component, modeled on the Linux `perf_event` subsystem, makes the assumption that values returned by `rdpmc` will always represent an integer and therefore it sign-extends the results via bit shifting. The code segment responsible is shown in Listing 4.

Listing 4. Sign Extension Code in PAPI `rdpmc` Read.

```

/* Read counter value */
pmc = rdpmc(index - 1);

/* sign extend result */
...
pmc <<= (64 - width);
pmc >>= (64 - width);

```

The value of “width” is drawn from the Linux `perf_event` subsystem, where on x86 machines it is set to the “bit width of general-purpose, performance monitoring counter” via a call to the `cpuid` instruction accessing the architectural performance monitoring leaf ($EAX = 0xA$) [18]. For the Raptor Lake microarchitecture, this is $width = 48$.

This means that during sign extension the upper 16 bits of the `PERF_METRICS` MSR are chopped off, making it impossible to actually read the memory bound and fetch latency metrics. The result is that in Listing 3 the `mem_bound` and `fetch_lat` variables are always set to 0, and their respective sibling metric values in `core_bound` and `fetch_band` are always equal to the parent metric’s value. Without modifying the PAPI `perf_event` component to remove sign extension, this method therefore leaves half of the level 2 TMA metrics with incorrect values.

V. EXPLICITLY SUPPORTING TMA IN PAPI

Using the PAPI library’s component system, support for collecting top-down metric measurements from the `PERF_METRICS` MSR is added without interfering with the core of the library.

A. Collecting Metrics

In general x86 MSRs can only be read using the `rdmsr` instruction which must be executed at privilege level 0. However the various performance MSRs can be read using the special `rdpmc` instruction which can be configured to allow reading at any privilege level. On Linux user access to `rdpmc` is allowed as long as a `perf_event` event is actively being used. To read a value `rdpmc` is executed with the `ECX` register set to `0x20000000`. The full 64-bit result is then extracted from the `EDX` and `EAX` registers.

Once the 64-bit `PERF_METRICS` value is read the Level 1 metrics and directly available Level 2 metrics can be converted by splitting out the fractional 8-bit values into percentages by Equation 1. In this equation, M_a and M_b are the metric’s 8-bit fractional value before and after the calipered code block, and S_a and S_b are the number of slots counted in fixed counter 3 before and after the calipered code block.

$$M_{\%} = \frac{\frac{M_b}{255}S_b - \frac{M_a}{255}S_a}{S_b - S_a} * 100 \quad (1)$$

For $S_a = 0$, this can be trivially simplified to remove the dependency of slots from the equation, as shown in Equation 2.

$$M_{\%} = \frac{M_b}{255} * 100 \quad (2)$$

The Level 2 metrics that are not directly available in the MSR can be derived from their parent and sibling metrics by Equation 3, where $P_{M_{\%}}$ and $S_{M_{\%}}$ are the metric percentages for the parent and sibling respectively.

$$M_{\%} = P_{M_{\%}} - S_{M_{\%}} \quad (3)$$

Ensuring that $S_a = 0$ so that Equation 2 can be used is simple. In `PAPI_start` a `PERF_EVENT_IOC_RESET` `ioctl()` call is sufficient to reset the slots counter. With a reset slots counter the `rdpmc` instruction only needs to be called once in `PAPI_stop`.

This strategy of resetting fixed counter 3 (slots) at the start of a measurement period is recommended by Intel’s software developer manual. The SDM notes that the `PERF_METRICS` MSR should start from zero as well, so we use the `ioctl` syscall to reset it in `PAPI_start` immediately after resetting fixed counter 3.

VI. HANDLING HETEROGENEOUS CPUs

On all of Intel’s heterogeneous multicore processors released to this date, the `PERF_METRICS` MSR only exists on the p-cores. This raises some issues.

Since the `rdpmc` instruction throws a general protection exception when attempting to access an unsupported counter, on heterogeneous processors it is important to check that the type of core the calipered program is running on supports the `PERF_METRICS` MSR to avoid encountering a segmentation fault. During component initialization if the target processor is heterogeneous a check is performed to ensure the program is executing on a supported core. If not, the PAPI `topdown` component is disabled and an error message supplied to the user explaining the problem.

This alone is insufficient, as Linux schedulers can change the processor affinity of running programs. If the calipered program is launched on a supported core and gets moved to an unsupported core during execution then attempting to collect top-down metrics will result in a segmentation fault. To the end user, especially those unused to the intricacies of heterogeneous processors, it is not immediately clear what caused the fault. It is important to check that the core type

is supported before each use of the `rdpmc` instruction and exit with a meaningful error message if the core type is unsupported.

A simple approach to this is to execute the `x86 cpuid` instruction indexing leaf 1AH directly before each call to `rdpmc`, read the core type it reports, and gracefully exit if the core type is unsupported. Unfortunately this is not always sufficient to prevent a segmentation fault. We will refer to this as the “simple” heterogeneous protection method.

A. Investigating Robustness

In order to test whether the protections implemented for heterogeneous cores are sufficient we conduct an experiment where a program using the PAPI `topdown` component is moved from a supported core to an unsupported core during its execution. The goal is to create absolute worst-case scenarios to catch potential errors.

Since the critical section containing `rdpmc` occurs in `PAPI_stop` we repeatedly call `PAPI_start()` and `PAPI_stop()` in a loop to increase the chance that migrating the program will bypass the core type check and result in a segfault. The script launching this program sleeps for a random period of time that is less than the execution time of the program, and then uses `taskset` to change its affinity from a supported core type (p-core) to an unsupported core type (e-core). We test on the system Raptor Lake M1 as described by Table I, repeating 1 million times. Every segmentation fault is considered an error, and every graceful exit a success.

With no protections in place, the error rate is 100%. Using the simple check introduced earlier in Section VI, it drops to only 0.0338%.

This worst-case scenario is not representative of most workloads, and the occasional failure is probably acceptable for most users. However it would be better to have a more robust solution, especially in potential scenarios such as a long-running multiple day experiment crashing due this problem without a meaningful error message.

B. Leveraging Restartable Sequences

When the Linux kernel mailing list was contacted about the issue with heterogeneous systems, they initially claimed it would not be easy or plausible to avoid the problem in the kernel. (Though later a different kernel developer decided that yes, it actually would be possible, and posted a patch that could avoid the GPF. However as of the publication of this paper the patch has not been merged). Initially it was suggested we could avoid the problem with restartable sequences.

Restartable sequences (rseq) were introduced to the Linux kernel in version 4.18 [19]. Rseq provides a way to create critical sections of user-space code that are effectively guaranteed to be executed sequentially. This is achieved through a per-thread memory region shared by both the user-space program and the kernel, taking the form of a structure that defines the critical section called the `rseq` structure. When the critical section defined by this structure is preempted or interrupted,

TABLE IV
PAPI SEGFAULT RATE FOR HETEROGENEOUS PROTECTION TEST ON MACHINE M_1 .

Protection type	Segfaults	Failure rate
None	10^6	100%
Simple	338	0.0338%
rseq	0	0%

the critical section immediately aborts, and can be restarted from the top.

While the use cases of rseq have previously focused on the safe, lock-free manipulation of per-cpu data, we can leverage it here to ensure that errors are handled gracefully even if the program being calipered by PAPI is moved to an unsupported core between the safety check and the actual execution of the `rdpmc` instruction. We use the `librseq` [20] library to integrate rseq into the PAPI `topdown` component.

A potential downside of using restartable sequences for this purpose is that it requires writing inline assembly. Each architecture PAPI wishes to support rseq protection on requires its own implementation. This downside does not apply here since the `topdown` component introduced in this article is intended only for x86 machines in the first place, but should be considered when adapting this method to other use cases.

The results for each method for protecting `rdpmc` on heterogeneous processors are shown in Table IV. As demonstrated, the protection method leveraging rseq is able to ensure that no segmentation faults can occur due to the unexpected migration of the calipered program from a supported core to an unsupported core.

VII. INVESTIGATING CYCLES OVERHEAD

In order to examine the performance overhead of the `topdown` component the CPU cycles for the two previous methods of collecting top-down metrics that were presented in Section IV are compared with the two variations of the new `topdown` component. The previous methods using the PAPI `perf_event` component with `rdpmc` disabled and `rdpmc` enabled are respectively referred to as methods A and B. The method enabled by the `topdown` component is tested both with rseq disabled and enabled, respectively referred to as methods C and D.

The overhead test consists of a simple loop that repeatedly calls `PAPI_start` and `PAPI_stop`. The clock cycles for each iteration of the loop are recorded. The test is reconfigured and run to properly benchmark each of the methods. Each configuration of the test is executed on the machines M_1 , M_2 , and M_3 , respectively described in Tables I.

Method D was unable to be tested on M_3 as while Linux 4.18.0 technically supports restartable sequences, it is poorly handled by `librseq`. The results are recorded in Table V and displayed in Figure 2.

While the pre-existing method B had significantly lower performance overhead than method A, as discussed in Section IV-2 it was not able to collect all level 2 TMA metrics

TABLE V
CYCLES OVERHEAD FOR N=100 MILLION ITERATIONS.

Test config.	Median cycles			Speedup w.r.t A		
	M_1	M_2	M_3	M_1	M_2	M_3
A (pe w/o rdpmc)	8487	8890	13986	-	-	-
B (pe w/ rdpmc)	3427	3813	5580	2.48x	2.33x	2.51x
C (td w/o rseq)	2705	2312	2132	3.14x	3.85x	6.56x
D (td w/ rseq)	2497	2106	-	3.40x	4.22x	-

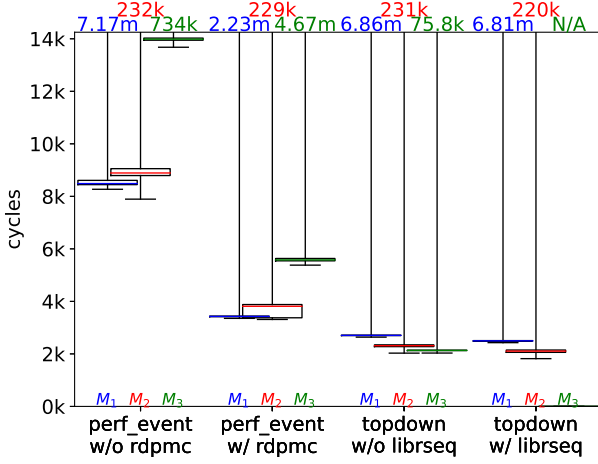


Fig. 2. A boxplot of the cycles overhead for each test executed on machines M_1 , M_2 , and M_3 . The whiskers represent minimum and maximum recorded cycle counts. Due to the extreme nature of the maximum cycle count outliers, the plot is cut off at the top and their values are recorded on the figure.

due to quirks with PAPI’s sign extension and is therefore not truly a valid solution.

The new methods introduced in this article, C and D, are both able to collect level 1 and 2 TMA metrics with greatly reduced cycle count overhead when compared to the most functional previous method.

The reduced cycle overhead seen in method D when compared to method C is not granted by the use of `librseq`, but a side effect of the requirement that restartable sequences must be written in inline assembly. Method C is implemented in plain C code, and simply happens to have not been written with performance optimization as the goal. On the other hand, method D is implemented largely in x86 assembly and as a result care was naturally taken to minimize the verbosity of the code (and as such the number of instructions). Since the difference in cycle overhead between methods C and D is due to implementation details, they should not be considered an effect of restartable sequences.

A. Outliers

A discussion on the extreme outliers seen in Figure 2 is warranted. There are noticeable cycle count outliers for the overhead test when configured to benchmark the `topdown` component with `librseq`.

Most of our overhead results lie near the median. However there are occasional outliers more than an order of magnitude over the median, with some reaching more than 1000x the

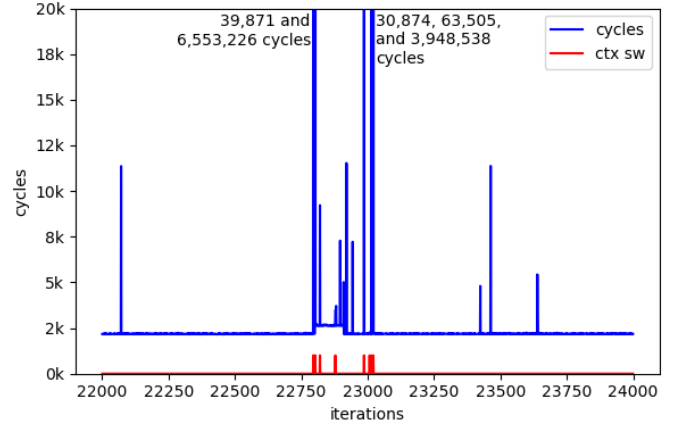


Fig. 3. Cycles and context switches per iteration of a run of the overhead test configured for the `topdown` component with `librseq` on machine M_1 from iteration 22,000 to iteration 22,4000. The larger outliers are caused by context switches.

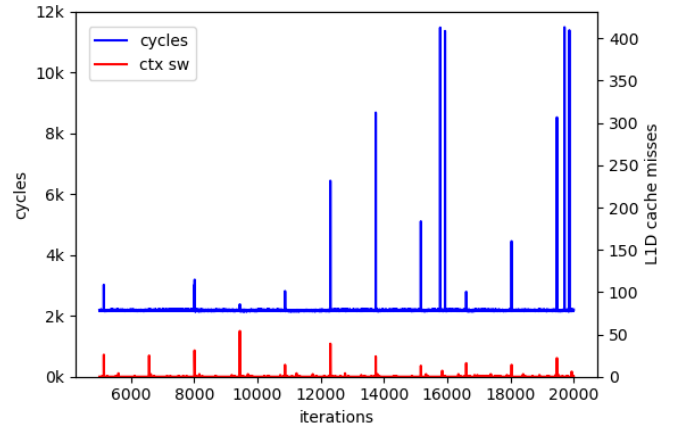


Fig. 4. Cycles and L1D cache misses per iteration of a run of the overhead test configured for the `topdown` component with `librseq` on machine M_1 from iteration 4,000 to iteration 20,000. Most outliers in the cycle overhead correspond to increased cache misses.

median. This behavior is similar to that observed when testing the overhead of PAPI with `rdpmc` in previous work [17]. A sample of these outliers is shown in Figure 3, which shows the cycle counts for a range of iterations of the overhead test.

Context switches account for over 90% of outliers that are at least an order of magnitude larger than the median cycle count. A portion of the overhead test results is plotted alongside measured context switches in Figure 3, demonstrating this correlation. Every vertical red line is a context switch.

The selected portion of results also contains some outliers at around 11k cycles, and some at around 5k cycles. These outliers cannot be attributed to context switches, but do correlate with L1D cache misses as shown in Figure 4.

Every 1430 iterations or so, the overhead test sees anywhere from 5 to 50 L1D cache misses.

These spikes in the cache miss rate align with most of the smaller spikes in the cycle overhead, but notably do not seem

to account for the larger spikes at around 11k cycles.

While we do observe page faults every 512 iterations due to storing measured cycle counts for the test itself as in previous work [17], in this case they do not correlate with any repeating patterns in the cycle count overhead. (The overhead test stores one 64-bit unsigned integer per iteration. 512 64-bit writes fill one 4096 byte page, causing the overhead test itself to induce regular page faults.) Further investigation was deemed out of scope for this work.

VIII. CONCLUSION AND FUTURE WORK

This paper discusses various ways PAPI is being enhanced to support easier gathering of Top-down metrics.

In addition it describes the design of the `topdown` component for the PAPI library to enable calibrated collection TMA metrics on Intel processors via the `PERF_METRICS` MSR with significantly lower performance overhead than the previous methods. Additionally, restartable sequences are used to guarantee that the `topdown` component never throws segmentation faults when an attempt to access the core-type specific `PERF_METRICS` MSR on a heterogeneous multicore processor, and in cases where `rseq` is not available a naive solution achieves a failure rate of only 0.0338%. This `topdown` component was contributed to PAPI and will be included in the next major PAPI release.

While the implementation described in this paper is sufficient for most workloads, the solution for preventing potential segmentation faults on heterogeneous processors it introduces is, while functional, unwieldily. Segmentation faults are avoided, but intentionally exiting a program with a useful error message is only so much better. A possible solution that would allow for programs which are moved between supported and unsupported cores without exiting would be to record the percentage of the program's runtime that top-down metrics were able to be collected for. However, this solution cannot be cleanly implemented without extensive modifications to the PAPI core, and is therefore left for future work.

ACKNOWLEDGMENT

This work was supported by the National Science Foundation under Grant No. CSSI-2311709.

The authors would like to thank Mathieu Desnoyers for his work on the creation and maintenance of `librseq` as well as the direct advice and feedback he provided which was instrumental to understanding and using restartable sequences. We would also like to thank Abdullah Al Nomaan Nafi and Heike Jagode for allocating their time and machines to repeating our overhead tests.

REFERENCES

- [1] A. Yasin, "A top-down method for performance analysis and counters architecture," in *Proc. IEEE International Symposium on Performance Analysis of Systems and Software*, Mar. 2014, pp. 35–44.
- [2] M. Jarus and A. Oleksiak, "Top-down characterization approximation based on performance counters architecture for amd processors," *Simulation Modelling Practice and Theory*, vol. 68, pp. 146–162, 2016.
- [3] C.-Y. Mou, C.-C. Hsiao, and J. Chou, "Top-down microarchitecture analysis approximation based on performance counter architecture for sive risc-v processors," in *International Workshop on RISC-V for HPC*, 2024, pp. 1666–1675.
- [4] J. Mundichipparakkal, D. Greene, J. Randall, V. Coubard, M. Williams, and R. Jong, "Multi-level TDA for heterofeneous arm processors using a standardized telemetry framework," in *Proc. IEEE International Symposium on Performance Analysis of Systems and Software*, Apr. 2026.
- [5] Arm, *Arm Topdown methodology v2.0*.
- [6] F. Banchelli, M. Garcia-Gasulla, and F. Mantovani, "Top-down models across CPU architectures: Applicability and comparison in a high-performance computing environment," *Information*, vol. 14, no. 10, 2023.
- [7] A. Saiz, P. Prieto, P. Abad, J. Gregorio, and V. Puente, "Top-down performance profiling on NVIDIA's GPUs," in *IEEE International Parallel and Distributed Processing Symposium*, 2022, pp. 179–189.
- [8] P. J. Mucci, S. Browne, C. Deane, and G. Ho, "PAPI: A portable interface to hardware performance counters," in *Proc. Department of Defense HPCMP User Group Conference*, Jun. 1999.
- [9] A. Kleen, "pmu-tools," 2024. [Online]. Available: <https://github.com/andikleen/pmu-tools/tree/master>
- [10] Intel. (2024) Intel VTune profiler performance analysis cookbook. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/cookbook/2025-0/overview.html>
- [11] Arm. (2024) Telemetry solution. [Online]. Available: <https://gitlab.arm.com/telemetry-solution/telemetry-solution>
- [12] A. Horvath, J. Slocum, and W. Zhang, "Memory bandwidth benchmark," <https://github.com/raas/mbw/tree/master>, 2006.
- [13] AMD, *Performance Monitor Counters for AMD Family IAh Model 50h-57h Processors*, Dec. 2025.
- [14] Intel, "Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 4: Model-specific Registers", 2024.
- [15] Intel., "Perfmon events," <https://github.com/intel/perfmon>, 2014.
- [16] *perf_event manual page*, 2024. [Online]. Available: https://man7.org/linux/man-pages/man2/perf_event_open.2.html
- [17] Y. Liu and V. M. Weaver, "Enhancing papi with low-overhead rdpmc reads," in *Programming and Performance Visualization Tools*, A. Bhatlele, D. Boehme, J. A. Levine, A. D. Malony, and M. Schulz, Eds. Cham: Springer International Publishing, 2019, pp. 3–20.
- [18] Intel, "Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 2: Instruction Set Reference, A-Z", 2024.
- [19] M. Desnoyers. (2018, 06) rseq: Introduce restartable sequences system call. [Online]. Available: <https://github.com/torvalds/linux/commit/d7822b1e24f2df5df98c76f0e94a5416349ff759>
- [20] —, "Library for restartable sequences," <https://github.com/compudj/librseq>, 2024.